

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享



High-Level IR Design in Programming Languages: A Case Study of Cangjie

Speaker: Tu Le





研讨/分享/感悟

Table of contents

- Background and Motivation
- Industry Best Practices
- Cangjie High-Level IR Design



研讨/分享/感悟

Table of contents

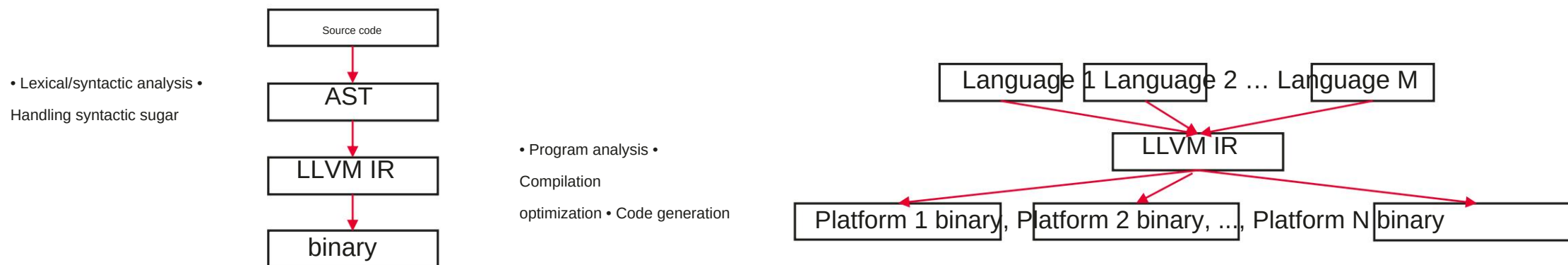
- Background and Motivation
- Industry Best Practices
- Cangjie High-Level IR Design



研讨/分享/感悟

Traditional compiler architectures typically include intermediate representations with two different levels of abstraction: AST and IR

- Taking the current mainstream LLVM compilation architecture as an example:



- LLVM IR, as the central link between the front-end and back-end, effectively solves the problem of fragmented compiler implementations across M languages and N platforms.
- The community has accumulated a wealth of technical assets around LLVM IR (numerous analysis and optimization algorithms, code generation implementations for different target platforms, etc.), and emerging programming languages often want to interface with LLVM IR to reuse these technical assets.



研讨/分享/感悟

There is a gap between the requirements of modern language design and traditional compiler architecture.

Gap 1: Stronger compile-time code inspection

Source code (CJ):

```
// helloWorld.cj func
foo(cond: Bool) { let idx = 100 if
    (cond) { idx = 100

}
let arr = [1, 2, 3, 4] return arr[idx]
}
```

Key

- points: • Analyze to find idx=100
- Understand that arr[100] is an array element access operation

Expected compilation result:

==> helloWorld.cj:7:12: | 3 | |

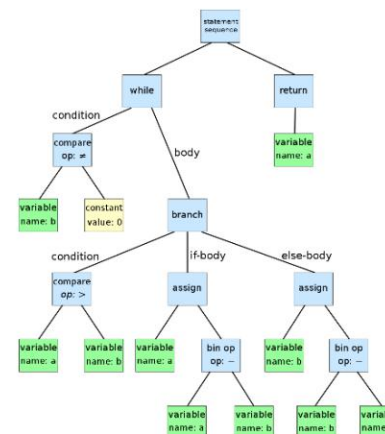
return arr[idx]

~~~~~

array

index idx is past the end of array (which contains 4 elements)

- An AST is a tree-like hierarchical structure that closely resembles the source code, making it difficult to perform data flow analysis.



- LLVM IR has data flow analysis capabilities, but it cannot perceive high-level semantics.

arr[idx] has been desaccharified into a function call on the AST.

Semantic behavior of functions that cannot be perceived on LLVM IR

```
%12 = call i64 @_ZN8std$score5Array1l2[]Ei"(i8* null,
%"record._ZN8std$score5Array1lE"* %arr, i64 1)
```





研讨/分享/感悟

There is a gap between the requirements of modern language design and traditional compiler architecture.

## Gap 2: Compilation optimization opportunities arising from higher levels of semantic information abstraction

Source code (CJ) and expected results:

// Note that in Cangjie, non-public classes are not publicly

visible. `open class Base

```
{ open func foo() { 0 }
```

```
}
```

open class Sub1 <: Base { open

```
func foo() { 1 }
```

```
}
```

func test(Sub1 obj) { // The

virtual function call here can be optimized to a direct call to

obj.foo() of Sub1::foo

```
}
```

func test(arr: Array<Int64>) { map(

```
}
```

Key

points: • Analyze the actual type of obj.

Sub1 •

Understanding that Base and Sub are not visible outside the

package • Understanding the virtual tables inside Base and Sub1

Allow

- There are no virtual tables in the AST; the relevant semantic information is implicit in the code.

- LLVM IR does not have a native concept of virtual tables, and there are generally two approaches:

First, the virtual table is implemented as a data structure supported by LLVM IR (such as an array), and the

virtual function call is implemented as element access and function pointer call within the data structure . At

**this point, the virtual function call optimization problem transforms into a constant propagation**

**analysis problem, losing the opportunity to combine other semantic information for optimization in some scenarios.**

After being converted to a virtual table in the example, `obj` is a function parameter. From the perspective of constant propagation analysis, its virtual table value is unpredictable. However, we can actually determine its virtual table content based on the fact that Sub1 cannot be further inherited .

```
%14 = getelementptr inbounds ptr, ptr %13, i64 0 %15 = load
ptr, ptr %14 %16 = call i32
%15(ptr %12)
```

2. Encapsulate the specific implementation of virtual tables and virtual function calls within a runtime-provided package.

Within the intrinsic, LLVM is unable to perceive the semantic information of the virtual tables and virtual function calls contained within the intrinsic.



研讨/分享/感悟

There is a gap between the requirements of modern language design and traditional compiler architecture.

### Gap 3: Implementation Requirements for Language-Specific Features

Source code (Rust) and expected results:

```
fn foo() { let
    r; let
    cond: bool = ...; let v =
    10; if cond { let
    x = 5; r =
        &x; } else { r
        = &v;

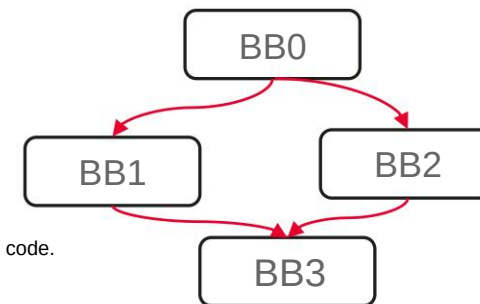
    }
```

```
// Error: r may be referencing a variable whose lifetime
has ended. println!("r: {r}");
```

Key

takeaways: • Data flow analysis  
capabilities • Lifecycle analysis  
capabilities (especially  
lifecycles caused by {} scope ) •  
Understanding Rust's ownership  
semantics (e.g.,  
borrow and move)

- An AST is a tree-like hierarchical structure that closely resembles the source code, making it difficult to perform data flow analysis.
- LLVM IR is a flattened structure ( CFG composed of BasicBlocks), and the scope hierarchy information caused by the curly braces {} in the source code has been lost.



The variable x is defined in BB1, but the flattened structure cannot reconstruct the scope caused by the curly braces {} in the source code.

- LLVM IR does not have the semantic ability to perceive rust ownership.



研讨/分享/感悟

**There's no problem that adding an intermediate layer can't solve; if there is, just add another la**

"All problems in  
computer science can be  
solved by another level  
of indirection."

David Wheeler







研讨/分享/感悟

# High-Level IR: Bridging the Gap Between AST and Low-Level IR

- The design philosophy and target benefits of High-Level IR are as follows:

The level of abstraction lies between AST and Low-Level IR:

Compared to Low-Level IR, it retains richer high-level semantic information for compilation optimization; and provides a more accurate source code location mapping for error alerting.

Compared to AST, the appropriate introduction of low-level semantic representations is beneficial for analysis and optimization; more stable representations (from the perspective of language evolution) are beneficial for engineering maintenance.

More explicit data flow/control flow information is beneficial for program analysis and compilation optimization.

Provides a platform for implementing language-specific features.

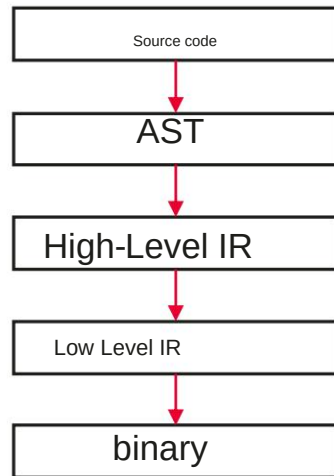
- Everything has a price, and the price of High-Level IR is:

More complex compilation front-end

> Increased compilation overhead (compilation time, compilation resource consumption)

• Lexical/syntactic analysis •  
Handling syntactic sugar

• Program analysis (low level) •  
Compiler optimization (low level)  
• Code generation



• Program analysis (high level) • Compiler optimization (high level) • Implementation of language-specific features



研讨/分享/感悟

## Table of contents

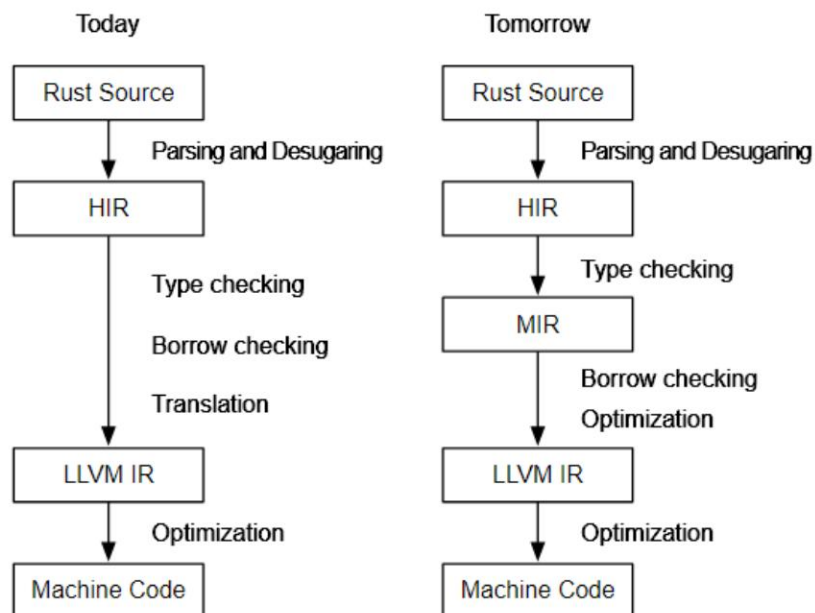
- Background and Motivation
- **Industry Best Practices**
- Cangjie High-Level IR Design



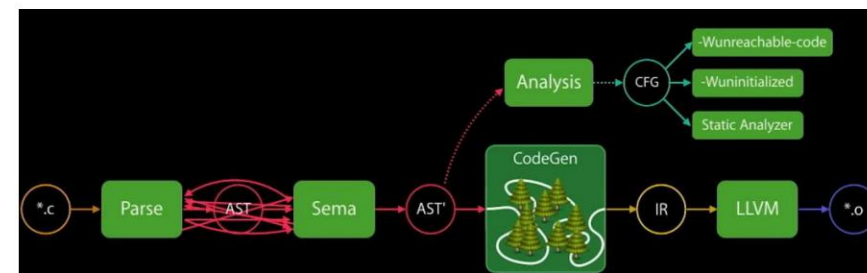
研讨/分享/感悟

# High-Level IR in Rust & Swift: MIR

MIR in Rust Compiler



SIL in Swift Compiler



[Introducing MIR | Rust Blog \(rust-lang.org\)](https://rust-lang.org/blog/2018-08-08-introducing-mir.html)

[The MIR \(Mid-level IR\) - Rust Compiler Development Guide \(rust-lang.org\)](https://rust-lang.org/blog/2018-08-08-introducing-mir.html)

[2015 LLVM Developers' Meeting: Joseph Groff & Chris Lattner "Swift's High-Level IR: A Case Study..." \(youtube.com\)](https://www.youtube.com/watch?v=8vXqYqYqYqY)



研讨/分享/感悟

## Table of contents

- Background and Motivation
- Industry Best Practices
- Cangjie High-Level IR Design



研讨/分享/感悟

# Cangjie High-Level IR (CHIR) Design Goals

- Resolves the semantic gap between AST and LLVM IR, **preserving more high-level semantic information for program analysis and compilation optimization compared to LLVM IR**. >

Semantic information contained in user-defined types in the Cangjie source code, such as type inheritance relationships and virtual table

information. > Semantic information contained in Cangjie expressions, such as loop variables, boundaries, and scope of code blocks in loop structures.

```
// CJ
source code
    class Base { open func foo() : Unit {...}
    }

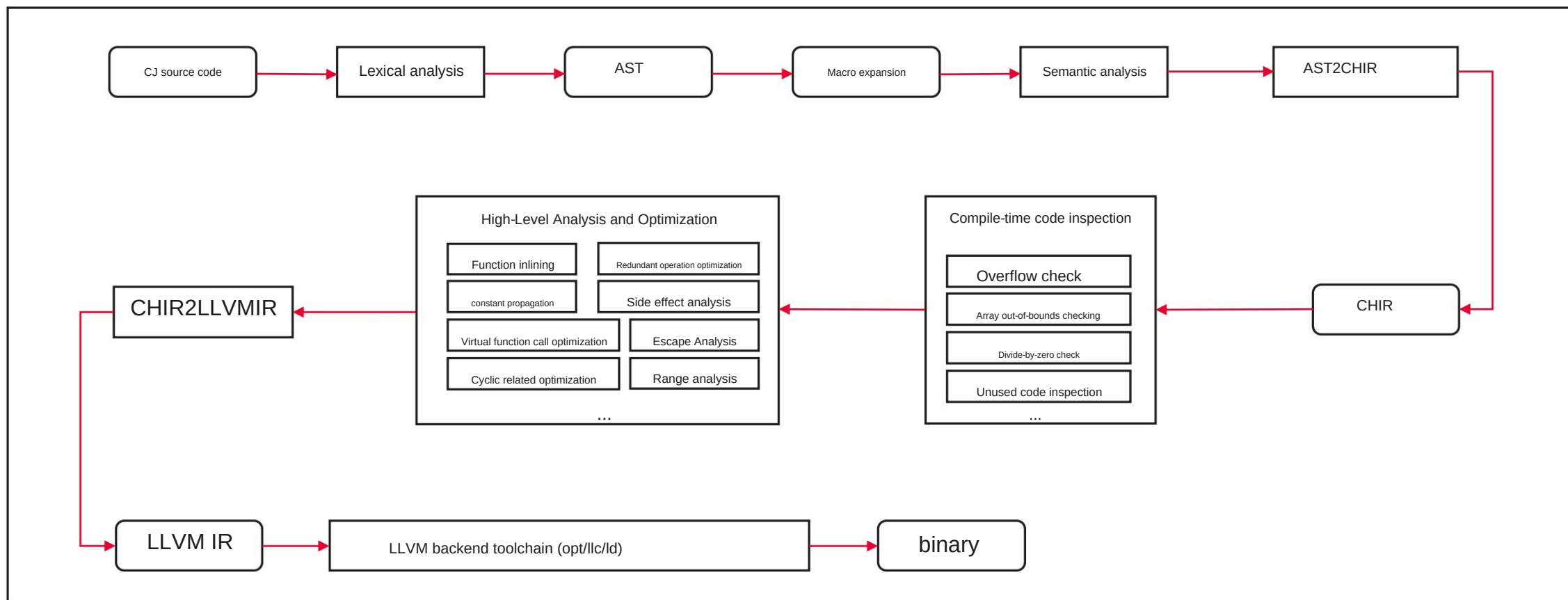
    class SubClassA <: Base
    { override func foo() : Unit {...}
    }
```

```
// CJ source
code var sum = 0
for (i in 0..100) { let innerVar =
    sum += i
    ...
}
```

- **Compared to AST, it provides data flow analysis capabilities, enabling program analysis and compilation optimization.**

- CHIR serves as a decoupling layer between the front-end and back-end, providing a general analysis and optimization platform that isolates syntax changes and is independent of back-end implementation.

## Overview of the Cangjie compilation process based on CHIR





研讨/分享/感悟

## CHIR Syntax Overview 1

```
v ::=
    | l |           // literal, including integer, float, char, bool and string etc
    x |           // variable, including global variable and local variable
    f |           // func identifier
    im |          // imported variable/func identifier
    t |           // type identifier
    b |           // block identifier
    blockGroup
```

```
statement ::=
    | terminatorStmt // statement which will trigger control-flow transition between blocks
    | trivialStmt    // other statements
```

// all statements inside will always be executed in order, last terminatorStmt will trigger control-flow // transist to successor blocks or exit the belonging blockGroup

```
block ::=
    b: trivialStmt, ..., trivialStmt, terminatorStmt
```

// a control-flow graph built by all blocks and their control-flow transition dependencies inside. Always  
// have a block as the entry of the control-flow graph

```
blockGroup ::=
    block, ..., block
```

```
func ::=
    'func' f(x1, ..., xn) : t
    blockGroup
```



# CHIR Example 1

Source code (CJ):

```
func foo(x: Int32) { if (x == 10)
    { return x + 10
    }
    return x * 2
}
```

CHIR:

```
Func @_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 { // loc: "helloCJ.cj"-1-1 [entry] Block #0:

    [ret] %1: Int32& = Allocate(Int32)
    GoTo(#1)
Block #1:
    %2: Int32 = Constant(10)
    %3: Bool = Equal(%0, %2)
    Branch(%3, #2, #3)
Block #2:
    %4: Int32 = Constant(10)
    %5: Int32 = Add(%0, %4)
    %6: Unit = Store(%5, %1)
    Exit
Block #3:
    GoTo(#4)
Block #4:
    %7: Int32 = Constant(2)
    %8: Int32 = Mul(%0, %7)
    %9: Unit = Store(%8, %2)
    Exit
}
```

Additional attribute information is represented using [] or //.

The Terminator statement triggers a control flow jump.

Statements within a block are executed sequentially.





研讨/分享/感悟

# Design highlight 1: High-Level semantic support (via BlockGroup nesting)

```
terminatorStmt ::=
```

```
| Exit |
| GoTo(b)
| Branch(v, b1, b2) | ...
```

```
trivialStmt ::= | x: t =
```

```
basicExpr | x: t = complexExpr
```

```
basicExpr ::= |
```

```
UnaryOp(v)
| BinaryOp(v1, v2)
| Apply(f, v1, ..., vn)
| Allocate(t)
| Store(v1, v2)
| Load(v)
| Invoke(f, t, v1, ..., vn)
| InstanceOf(v, t) | ...
```

```
complexExpr ::= |
```

```
If(v, blockGroup1, blockGroup2)
| Loop(v, blockGroup)
| ForIn(v1, v2, blockGroup1, blockGroup2, blockGroup3)
| Lambda(x1, ..., xn => blockGroup)
```

CHIR introduces statements that can express high-level semantics, which helps retain more semantic information:

For example, ``Invoke`` and ``InstanceOf`` can hide the details of underlying virtual tables and object memory layout, directly representing higher-level semantic information closer to the source

code. Meanwhile, native ``If``, ``Loop``, ``ForIn``, and ``Lambda`` statements support nested ``BlockGroup`` statements, preserving more of the code structure and syntactic sugar information naturally present in the Cangjie source code. However, it is particularly important to note that **expressions supporting nested ``BlockGroup`` statements must not violate the principle that "statements within a block are executed strictly in sequence,"** otherwise it will complicate control flow analysis.



研讨/分享/感悟

## CHIR Example 2

### Source code (CJ):

```
func foo(x: Int32) { if (x == 10)
    { return x + 10
    }
    return x * 2
}
```

Note that, in order to ensure the sequential execution of statements in Block #1, the return statement in the true branch of the If statement is actually delayed and executed outside.

### CHIR:

Func @\_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 { [entry] Block #0:

```
[ret] %1: Int32& = Allocate(Int32)
GoTo(#1)
```

Block #1:

```
%2: Int32 = Constant(10)
%3: Bool = Equal(%0, %3)
%4: Int32& = Allocate(Int32)
```

```
If (%3, {
    Block #2:
        %5: Int32 = Constant(10)
        %6: Int32 = Add(%0, %5)
        %7: Unit = Store(%6, %1)
        %8: Int32 = Constant(1)
        %9: Unit = Store(%8, %4)
        Exit
    }, {
```

```
    Block #3:
        Exit
    })
```

```
%10: Int32 = Load(%4)
%11: Int32 = Constant(0)
%12: Bool = Equal(%10, %11)
Branch(%12, #5, #4)
```

Block #4:

```
Exit
```

Block #5:

```
%13: Int32 = Constant(2)
%14: Int32 = Mul(%0, %13)
%15: Unit = Store(%14, %1)
Exit
```

```
}
```

The If statement supports nested BlockGroups, thus preserving the scope information of the If expression in the source code.



## Supports changes and benefits after nesting BlockGroups

After introducing nested BlockGroups: •

Statements within a Block will always execute strictly in sequence.

• The control flow relationship between Blocks forms a CFG (Containment

Flow Group) . • **Some statements will enter its nested state during execution.**

**Within a sub-CFG composed of BlockGroups, the sub-CFG guarantees that**

**control flow is returned to the statement, and then execution continues sequentially downwards.**

**Benefits:**

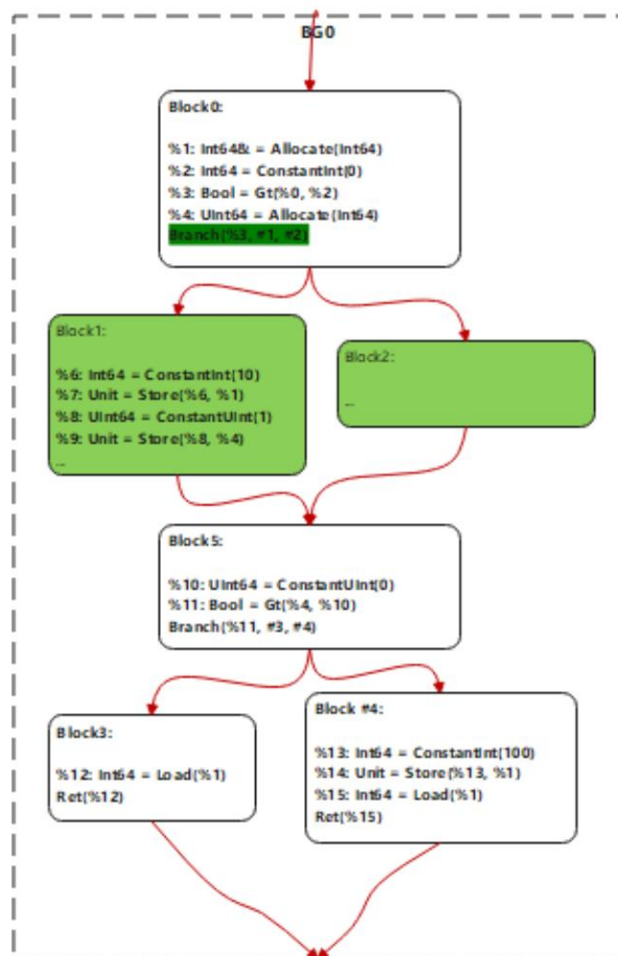
• BlockGroup itself can host the source code

Scope semantic information; inner BG can access outer BG.

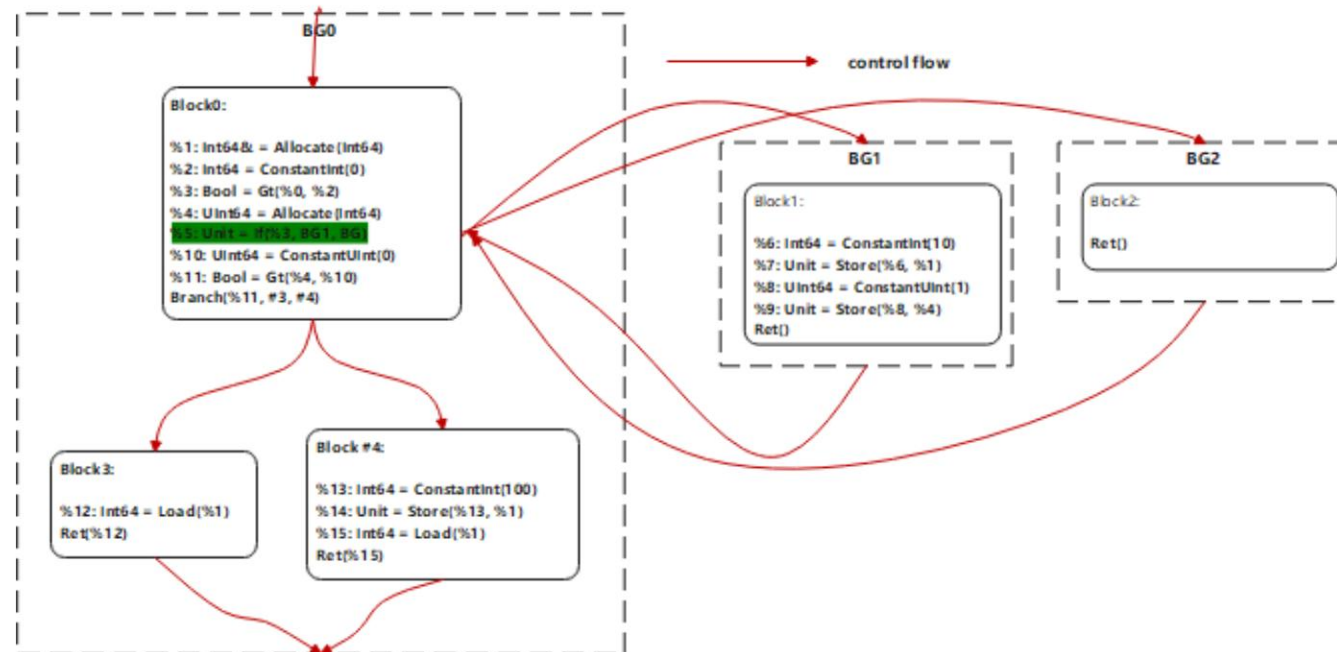
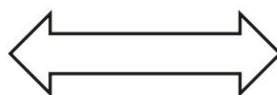
The value of BG, and vice versa.

• Instructions that can express more complex semantics (such as for-in)

and be designed in IR based on nested BlockGroups.



Equivalent transformations can be performed between the two.





研讨/分享/感悟

## CHIR Grammar Overview II

```
// declaration which defines a imported function or global variable from other packages importDecl ::= '#from' package_name 'import' im : t
```

```
// declaration which defines a user-customized type, including enum, struct and class customTypeDecl ::= |  
enumDecl | structDecl  
| classDecl
```

```
// declaration which defines a global variable globalVarDecl ::= v : t
```

```
// the basic compilation unit
```

```
Package ::=  
  {importDecl}  
  {customTypeDecl}  
  {globalVarDecl}  
  {func}
```



## Design Point Two: Retain the semantic information of Cangjie's custom types

### Source code (CJ):

```
class Base {
  static let s : Int64 = 0 let x : Int64

  var y : Int64

  init() {...} open
  func foo() : Unit {...} static func goo() :
  Unit {...} func koo() : Unit {...}
}

class SubClassA <: Base
{ override func foo() : Unit {...}
}
```

### CHIR:

```
[static] [readOnly] @_ZN11default23Base1s : Int64&
```

```
class @_ZN7default4Base: Class-Base {
```

```
  [static] let @_ZN11default23Base1s : Int64 let x : Int64
```

```
  var y : Int64 func
```

```
  @_ZN11default23Baseinit : (Class-Base &) -> Unit func
```

```
  @_ZN11default23Basefoo : (Class-Base &) -> Unit [static] func
```

```
  @_ZN11default23Basegoo : () -> Unit func @_ZN11default23Basekoo :
  (Class-Base &) -> Unit vtable { foo : (Class-Base &)->Unit =>
```

```
  @_ZN11default23Basefoo Preserves type inheritance relationship
}
```

```
} class @_ZN11default23SubClassA : Class-SubClassA <: @_ZN7default4Base { func
```

```
  @_ZN11default23SubClassAfoo : (SubClassA &) -> Unit vtable { foo : (Class-
  Base &)-
```

```
    >Unit => @_ZN11default23SubClassAfoo
  }
```

```
func @_ZN11default23Baseinit : (Class-Base &) -> Unit {...} func
```

```
@_ZN11default23Basefoo : (Class-Base &) -> Unit {...} [static] func
```

```
@_ZN11default23Basegoo : () -> Unit {...}
```

```
...
```

Reserved type member variable  
layout information + static  
member variable information

Methods in a custom type  
are translated into global  
functions while preserving  
their relationship with the  
custom type itself.

Native support for vtable  
information recording



## Some future career directions

- In conjunction with the design of the instruction and type system, express other high-level semantics to facilitate analysis and optimization, such as: box/unbox,
- Better support for the innovative features that Cangjie is exploring and designing.
- As part of the future Cangjie open ecosystem, it provides ecosystem developers with a scalable program analysis and compilation optimization platform.

 **WORKSHOP** 研讨/分享/感悟  
仓颉社区中的灵感碰撞 / 社区先行者的干货分享



# Thank you for watching!



Follow the "Cangjie Programming Language" official WeChat account to get the latest official information!





Before "internal connection":

Func @\_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 { [entry] Block #0:

[ret] %1: Int32& = Allocate(Int32)  
GoTo(#1)

Block #1:

%2: Int32 = Constant(10)  
%3: Bool = Equal(%0, %3)  
%4: Int32& = Allocate(Int32)

If (%3, {  
    Block #2:  
        %5: Int32 = Constant(10)  
        %6: Int32 = Add(%0, %5)  
        %7: Unit = Store(%6, %1)  
        %8: Int32 = Constant(1)  
        %9: Unit = Store(%8, %4)  
        Exit  
}, {  
    Block #3:  
        Exit  
})

%10: Int32 = Load(%4)  
%11: Int32 = Constant(0)  
%12: Bool = Equal(%10, %11)  
Branch(%12, #5, #4)

Block #4:

Exit

Block #5:

%13: Int32 = Constant(2)  
%14: Int32 = Mul(%0, %13)  
%15: Unit = Store(%14, %1)  
Exit

}

The If statement is converted into a  
Branch directive and two inlined  
Blocks. The Exit directive within the Block  
is replaced with a GoTo statement in the  
code following the If statement.

After "internal connection":

Func @\_ZN7default3fooEi([readOnly] %0: Int32) : (Int32) -> Int32 { [entry] Block #0:

[ret] %1: Int32& = Allocate(Int32)  
GoTo(#1)

Block #1:

%2: Int32 = Constant(10)  
%3: Bool = Equal(%0, %3)  
%4: Int32& = Allocate(Int32)

Branch(%3, #2, #3)  
Block #2:  
    %5: Int32 = Constant(10)  
    %6: Int32 = Add(%0, %5)  
    %7: Unit = Store(%6, %1)  
    %8: Int32 = Constant(1)  
    %9: Unit = Store(%8, %4)  
    GoTo(#6)  
Block #3:  
    GoTo(#6)

Block #6:

%10: Int32 = Load(%4)  
%11: Int32 = Constant(0)  
%12: Bool = Equal(%10, %11)  
Branch(%12, #5, #4)

Block #4:

Exit

Block #5:

%13: Int32 = Constant(2)  
%14: Int32 = Mul(%0, %13)  
%15: Unit = Store(%14, %1)  
Exit

}

If Block #6 is further copied into #2  
and #3, the read/write and decision  
operations of the additional variable  
%4 can be eliminated through  
constant propagation.